

05 · Referencia técnica

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 05-technical-reference.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Las 36 acciones registradas
2. Funciones centrales del motor
3. Endpoints HTTP
4. Comandos
5. Variables de entorno
6. Campos del manifest
7. Errores y excepciones

Catálogo de consulta: las 36 acciones, las funciones relevantes del motor, los endpoints HTTP, los comandos, las variables de entorno, los operadores de condición y los errores del sistema. Para cada función relevante: firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién la llama, a quién llama y **riesgo al modificarla**.

1. Las 36 acciones registradas

Fuente: `engine/action_registry.py::_BUILT_IN_ACTIONS`. La columna **Flows** indica en cuántos de los 27 manifests aparece la acción.

1.1 `filesystem.*` — 7 acciones

Acción	Función	Firma	Retorno	Flows
<code>filesystem.ensure_directory</code>	<code>ensure_directory</code>	<code>(path: str)</code>	<code>{path, exists}</code>	0
<code>filesystem.list_directory</code>	<code>list_directory</code>	<code>(path: str, recursive: bool = False)</code>	<code>{path, files[], total_files}</code>	2
<code>filesystem.write_json</code>	<code>write_json</code>	<code>(path: str, data: Any)</code>	<code>{path, written}</code>	16
<code>filesystem.read_text_file</code>	<code>read_text_file</code>	<code>(path: str, max_chars: int = 4000)</code>	<code>{path, chars, preview}</code>	0
<code>filesystem.classify_file</code>	<code>classify_file_inventory</code>	<code>(files: list[dict])</code>	<code>{total_files, total_size_bytes, by_extension, largest_file}</code>	2
<code>filesystem.summarize_text</code>	<code>summarize_text_folder</code>	<code>(path: str, max_files: int = 10, max_chars_per_file: int = 800)</code>	<code>{path, processed_files, summaries[]}</code>	1
<code>filesystem.move_file</code>	<code>move_file</code>	<code>(source_path: str, destination_path: str, overwrite: bool = False)</code>	<code>{source, destination, moved}</code>	0

Excepciones: `list_directory`, `read_text_file`, `summarize_text_folder` y `move_file` levantan `FileNotFoundError` si la ruta no existe. `move_file` levanta `FileExistsError` si el destino existe y `overwrite=False`.

Efectos secundarios: `ensure_directory`, `write_json` y `move_file` crean el directorio padre con `mkdir(parents=True, exist_ok=True)`. `move_file` con `overwrite=True` hace `destination.unlink()` antes de mover: si el `shutil.move` falla después, el destino ya se perdió. `write_json` sobrescribe sin avisar.

Riesgo al modificar: `write_json` es la acción más usada del catálogo (16 de 27 flows). Cualquier cambio en su forma de retorno rompe el resumen del panel (`_smart_summary`) y la detección de outputs.

`summarize_text_folder` solo procesa extensiones `.txt`, `.md`, `.log`, `.csv`, `.json` y **corta en `max_files`** (10 por defecto). Un archivo fuera de esa lista se ignora en silencio: el resultado no dice cuántos se saltaron. `list_directory` **solo devuelve archivos**, nunca subcarpetas.

1.2 `screen.*` — 3 acciones

Acción	Función	Firma	Retorno	Flows
--------	---------	-------	---------	-------

<code>screen.capture_screenshot</code>	<code>capture_screenshot</code>	<code>(output_path: str)</code>	<code>{image_path, width, height, method}</code>	4
<code>screen.capture_region</code>	<code>capture_region</code>	<code>(output_path: str, bbox: dict)</code>	<code>{image_path, width, height, bbox, method}</code>	1
<code>screen.capture_active_window</code>	<code>capture_active_window</code>	<code>(output_path: str)</code>	<code>igual + window_title</code>	1

Estrategia de respaldo: las tres intentan primero `mss` y, si falla, `Pillow (ImageGrab)`. El campo `method` del retorno dice cuál se usó. Si ambas fallan, levantan `RuntimeError` con el mensaje «No fue posible capturar la pantalla. Revisa si el entorno tiene escritorio gráfico disponible».

Formato del `bbox` (`_resolve_bbox`): acepta `{left, top, width, height}` o `{left, top, right, bottom}`. Valores negativos en `left/top` se interpretan **relativos al borde opuesto, estilo CSS** — así el flow 19 captura la barra de tareas con `{"left": 0, "top": -48, "width": 99999, "height": 48}`, donde el ancho desmedido se recorta solo al borde del monitor. Un `bbox` que produzca dimensiones ≤ 0 levanta `ValueError`.

`capture_active_window` resuelve el rectángulo con `pygetwindow.getActiveWindow()`. Si no hay ventana identificable (sesión sin foco) levanta `RuntimeError` con motivo legible. **Solo captura el monitor primario** (`sct.monitors[1]`): una ventana en un segundo monitor daría un recorte incorrecto. **REQUIERE VALIDACIÓN**.

Riesgo al modificar: cambiar el orden `mss` → `Pillow` alteraría el color de las capturas en algunos equipos; `mss` devuelve BGRA y `mss.tools.to_png` lo maneja.

1.3 `vision.*` — 5 acciones

Acción	Función	Firma resumida	Flows
<code>vision.analyze_image</code>	<code>analyze_image</code>	<code>(image_path: str, analyzer: str = 'mock')</code>	1
<code>vision.ocr_image</code>	<code>ocr_image</code>	<code>(image_path: str)</code>	2
<code>vision.find_text_in_image</code>	<code>find_text_in_image</code>	<code>(image_path: str, query: str, case_sensitive: bool = False)</code>	0
<code>vision.select_image</code>	<code>select_image</code>	<code>(image_path: str)</code>	0
<code>vision.inspect_screen_target</code>	<code>inspect_screen_target</code>	12 parámetros; ver §1.9	0

`analyze_image` despacha sobre el diccionario `ANALYZERS` (`mock`, `metadata`, `ocr`) y levanta `ValueError` si el nombre no existe. Las cinco levantan `FileNotFoundError` si la imagen no está.

`ocr_image` **no falla cuando falta OCR:** `OCRImageAnalyzer` comprueba `pytesseract` y el binario `tesseract` (incluidas las rutas típicas de Windows fuera del PATH) y devuelve `status: "unavailable"` con `reason` y una `summary` con instrucciones por sistema operativo. Es una degradación deliberada, documentada en el propio archivo: permite que un flow con rama de recuperación siga su camino alternativo.

Riesgo al modificar: `find_text_in_image` e `inspect_screen_target` dependen de la clave `matches` que produce `OCRImageAnalyzer`, con `left/top/width/height` por palabra. Cambiar esa forma rompe el encadenamiento OCR → `ui.click_bbox`.

1.4 `system.*` — 6 acciones

Acción	Función	Firma	Flows
<code>system.wait_seconds</code>	<code>wait_seconds</code>	<code>(seconds: float)</code>	4
<code>system.snapshot_system</code>	<code>snapshot_system</code>	<code>()</code>	1

<code>system.top_processes</code>	<code>top_processes</code>	<code>(limit: int = 10, sort_by: str = 'memory')</code>	1
<code>system.watch_processes</code>	<code>watch_processes</code>	<code>(processes: list[dict], memory_mb_threshold: float = 250.0, cpu_percent_threshold: float = 60.0)</code>	1
<code>system.read_clipboard</code>	<code>read_clipboard</code>	<code>(max_chars: int = 10000)</code>	1
<code>system.run_powershell</code>	<code>run_powershell</code>	<code>(command: str, allowlist: list[str] \ None = None, timeout_seconds: float = 30.0)</code>	1

`snapshot_system` mide CPU con `psutil.cpu_percent(interval=0.2)`: **bloquea 200 ms** en cada llamada. El disco se mide sobre `psutil.disk_usage("/")`, que en Windows resuelve a la unidad del directorio actual.

`top_processes` acepta `sort_by` con valor `"cpu"`; **cualquier otro valor ordena por memoria**, sin error ni aviso. Los procesos que lanzan `NoSuchProcess` o `AccessDenied` se saltan en silencio, así que `total_seen` puede ser menor que el número real de procesos del equipo.

`read_clipboard` **nunca lanza excepción**: si falta `pyperclip` o no hay backend de portapapeles devuelve `{available: False, reason: "...", text: "", length: 0}`. Cuando trunca lo declara con `truncated: True` y `length` con el tamaño real.

`system.run_powershell` — la acción de mayor superficie

```
def run_powershell(command: str,
                  allowlist: list[str] | None = None,
                  timeout_seconds: float = 30.0) -> dict[str, Any]
```

Dos controles en cascada, ambos **antes** de invocar PowerShell:

1. **Tokens prohibidos.** Si el comando contiene `;`, `|`, `&`, ```, `>`, `<`, `$` (o `$_` → `ValueError`. Cierra el encadenamiento y la redirección.
2. **Allowlist de verbos.** La **primera palabra** del comando debe coincidir exactamente (sensible a mayúsculas) con un elemento de la lista. Sin `allowlist` explícita se usa `_PS_DEFAULT_ALLOWLIST`, 13 verbos de solo lectura: `Get-Date`, `Get-Process`, `Get-Service`, `Get-ComputerInfo`, `Get-CimInstance`, `Get-WmiObject`, `Get-Disk`, `Get-Volume`, `Get-NetAdapter`, `Get-NetIPAddress`, `Get-EventLog`, `Get-Host`, `Get-Location`.

Se ejecuta con `subprocess.run(["powershell", "-NoProfile", "-NonInteractive", "-Command", trimmed], shell=False)`. Devuelve `stdout` recortado a 50 000 caracteres y `stderr` a 5 000, **declarando el recorte** con `stdout_truncated` / `stderr_truncated`.

Riesgo al modificar: alto. El parámetro `allowlist` es *overridable desde el manifest*. Un flow que declare `"allowlist": ["Remove-Item"]` obtiene exactamente eso. La seguridad de esta acción depende de que nadie escriba ese manifest, no de un control del motor. Ver 11 · Seguridad.

1.5 ui.* — 7 acciones

Acción	Función	Firma	<code>dry_run</code>	Flows
<code>ui.open_url</code>	<code>open_url</code>	<code>(url: str, new_tab: bool = True)</code>	[NO]	1
<code>ui.open_file_in_browser</code>	<code>open_file_in_browser</code>	<code>(path: str, new_tab: bool = True)</code>	[NO]	0

<code>ui.launch_process</code>	<code>launch_process</code>	<code>(command: str, wait_seconds: float = 0.0, shell: bool = False, dry_run: bool = False)</code>	[OK]	2
<code>ui.hotkey</code>	<code>hotkey</code>	<code>(keys: list[str], interval: float = 0.0, dry_run: bool = False)</code>	[OK]	6
<code>ui.type_text</code>	<code>type_text</code>	<code>(text: str, interval: float = 0.01, dry_run: bool = False)</code>	[OK]	2
<code>ui.click</code>	<code>click</code>	<code>(x, y, clicks=1, interval=0.0, button='left', dry_run=False)</code>	[OK]	0
<code>ui.click_bbox</code>	<code>click_bbox</code>	<code>(bbox: dict, clicks=1, interval=0.0, button='left', dry_run=False)</code>	[OK]	0

dry_run es el mecanismo de prueba de todo el bloque de UI. Con **dry_run=True** la función devuelve el payload que habría producido, con **sent/typed/clicked** en **False**, sin tocar teclado ni ratón. Es lo que permite que `tests/test_actions_basic.py` las ejecute sin efectos. Todos los flows de UI exponen **dry_run** en su `context.example.json`.

launch_process prohíbe **shell=True** explícitamente, con un comentario que explica la decisión:

```
# Se elimina deliberadamente la rama shell=True para cerrar el vector de
# command injection (CWE-78).
raise ValueError('launch_process: shell=True está deshabilitado por seguridad. ...')
```

El comando se tokeniza con `shlex.split` y se lanza con **shell=False**. **Nota para Windows:** `shlex` usa reglas POSIX, así que una ruta con contrabarras (`C:\Users\x`) puede tokenizarse de forma inesperada. Los flows 10 y 13 pasan rutas con barra normal.

`ui.hotkey`, `type_text`, `click` y `click_bbox` importan `pyautogui` dentro de la función (`_import_pyautogui`) y levantan `RuntimeError` con mensaje explicativo si no está disponible o el entorno no permite control de UI.

Riesgo al modificar: muy alto. Estas acciones mueven teclado y ratón sobre la sesión real del operador. Un cambio que ignore **dry_run** haría que la suite de tests empiece a teclear en la máquina de quien la ejecute.

1.6 http.* — 2 acciones

Acción	Función	Firma	Flows
<code>http.fetch_url</code>	<code>fetch_url</code>	<code>(url: str, output_path: str \ None = None, timeout: float = 15.0)</code>	0
<code>http.check_urls</code>	<code>check_urls</code>	<code>(urls: list[str], timeout: float = 10.0, max_urls: int = 100, delay_seconds: float = 0.0)</code>	1

`fetch_url` hace `raise_for_status()`: un 404 se convierte en excepción y el paso falla.

`check_urls` verifica **en orden de entrada** (determinista):

- `http/https` → **HEAD** con redirects; si el servidor devuelve 405 o 501, reintentará con **GET** en modo **stream** y lo cierra. `ok = response.ok`.

- `file://` → comprueba existencia del archivo local. Sirve para auditar los enlaces de las páginas de demo sin red.
- Otros esquemas (`mailto:`, `tel:`) → `skipped: True`, no cuentan como rotos.

Devuelve `truncated: True` si la lista superaba `max_urls`. **Nunca trunca en silencio.**

Riesgo al modificar: el flow 24 lee `broken_count` para decidir si alertar. Cambiar la semántica de `ok` (por ejemplo tratando un 301 como roto) cambiaría el comportamiento del flow sin tocar su manifest.

1.7 `notify.send` — 1 acción

```
def send_notification(message: str, backend: str = 'log', target: str | None = None,
                    token: str | None = None, extra: dict | None = None,
                    timeout: float = 10.0) -> dict[str, Any]
```

backend	Requiere <code>target</code>	Efecto
<code>log</code>	No	<code>print</code> a stdout
<code>file</code>	Sí (ruta)	Añade <code>timestamp\tmessage\n</code> al archivo, creando el directorio
<code>webhook</code>	Sí (URL)	POST JSON <code>{"text": message, "timestamp": ...}</code> más <code>extra</code>

Un `backend` desconocido levanta `ValueError`. Falta de `target` en `file` o `webhook` también.

Resolución de secretos: si `token` empieza por `@secret:`, el resto se resuelve con `engine.secrets.get_secret`. El token nunca se escribe en el resultado: `record` devuelve `backend`, `message`, `timestamp`, `sent`, `target` y `status_code`, **no el token**. Es un detalle deliberado y correcto, porque ese resultado va al contexto y de ahí a la columna `context_json` de SQLite.

Usada por los flows 23 y 26, ambos con `backend: "file"` por defecto.

1.8 `browser.*` — 4 acciones

Acción	Función	Módulo	Flows
<code>browser.capture_page</code>	<code>capture_page</code>	<code>actions/browser_capture.py</code>	3
<code>browser.fill_form</code>	<code>fill_form</code>	<code>actions/browser_form.py</code>	1
<code>browser.extract_content</code>	<code>extract_content</code>	<code>actions/browser_extract.py</code>	6
<code>browser.crawl_site</code>	<code>crawl_site</code>	<code>actions/browser_extract.py</code>	1

Las cuatro importan `playwright.sync_api.sync_playwright` dentro de la función y levantan `RuntimeError` con el comando de instalación si falta. Las cuatro aceptan una URL (`http`, `https`, `file`) o una **ruta local a un `.html`**, que `_to_url` convierte a `file://` tras comprobar que existe; si no existe y no es URL, `FileNotFoundError`.

`browser.capture_page`

```
capture_page(target, output_path, full_page=True, viewport_width=1280,
            viewport_height=800, wait_seconds=1.0, timeout_seconds=30.0)
```

Chromium **headless**. Devuelve `{image_path, url, title, width, height, full_page, size_bytes, method}`. `width/height` son los del `viewport solicitado`, no los del PNG resultante: con `full_page=True` la imagen es más alta. Detalle que conviene conocer al leer los reportes.

`browser.fill_form`

```
fill_form(target, seeds_path='data/seeds/form_seeds.json',
         used_path='data/seeds/used_indices.json', headless=False,
         slow_mo_ms=250, save_data_path=None, viewport_width=1280,
```

```
viewport_height=900, timeout_seconds=30.0)
```

headless=False por defecto: lanza una ventana de Chromium visible. Rellena nueve campos con `page.fill` más `#pais` con `select_option`, hace clic en `#btn-submit` y espera `#validation-result.show` durante 5 s.

Selectores acoplados al HTML de demo. Los nombres de campo están escritos en el código (`nombre`, `apellido`, `email`, `telefono`, `direccion`, `ciudad`, `fecha_nacimiento`, `profesion`, `comentario`, `pais`). Apuntar esta acción a otro formulario no funciona sin tocar Python. Es una limitación real del alcance actual.

Éxito determinado por texto: `is_success` se calcula como `validation_text.startswith('[OK]')` or `'válido' in validation_text.lower()`. Depende del texto que renderiza la página de demo.

Efecto secundario persistente: `_pick_record` escribe `data/seeds/.used_indices.json` en cada llamada, incluso si el llenado falla después. Cuando los 100 registros se agotan, el tracking se reinicia solo y lo declara con `cycle_resetted: True`.

Aleatoriedad: `random.choice` sin semilla. Es la única fuente de no-determinismo del sistema, y es intencional (el objetivo es no repetir registros).

`browser.extract_content`

```
extract_content(target, selector=None, parse_number=False, include_tables=False,
                max_links=200, max_text_chars=200_000, track_state_path=None,
                save_markdown_path=None, save_tables_dir=None,
                viewport_width=1280, viewport_height=800,
                wait_seconds=1.0, timeout_seconds=30.0)
```

La acción más versátil del repositorio: seis flows la usan con parámetros distintos. Devuelve siempre `url`, `title`, `text`, `text_chars`, `text_truncated`, `content_hash` (SHA-256 del texto normalizado), `links`, `link_urls`, `links_count`, `links_truncated`, `meta`, `method`. Con `selector` añade `selector_found`, `selector_value` y —si `parse_number`— `selector_number`. Con `include_tables` añade `tables` y `tables_count`. Con `track_state_path` añade `first_run`, `changed`, `previous_value`, `state_path`.

`browser.crawl_site`

```
crawl_site(start_url, max_pages=10, max_depth=2, same_domain_only=True,
            delay_seconds=0.5, respect_robots=True, viewport_width=1280,
            viewport_height=800, wait_seconds=0.5, timeout_seconds=30.0)
```

BFS determinista. Devuelve `pages[]` con `url`, `depth`, `title`, `links_count`, `text_chars`, `content_hash`, más `truncated`, `robots_blocked`, `errors` y `robots_checked_hosts`.

Riesgo al modificar: alto en `browser_extract.py`. Sus funciones puras (`normalize_text`, `content_hash`, `resolve_links`, `parse_number`, `apply_tracking`, `crawl_pages`, `build_result`, `render_markdown`, `save_tables_csv`) concentran 31 de los 150 tests. Cambiar `normalize_text` cambia todos los hashes ya guardados y haría que el flow 23 reporte un cambio falso en la primera corrida posterior.

1.9 `rules.evaluate` y la acción de visión avanzada

```
evaluate_rules(input_data: dict, rules: list[dict], default_status: str = 'no_match')
```

Evalúa las reglas **en orden y se detiene en la primera que coincide**. Cada regla lleva `path`, `operator` (`eq`, `ne`, `gt`, `lt`, `contains`, `in`), `value`, y opcionalmente `id`, `status`, `status_on_fail`, `message`. Devuelve `{status, matched_rule, evaluations[]}`. Un operador no soportado levanta `ValueError`.

Cuidado: dos motores de condición distintos. `actions/rules.py::_matches` soporta **6 operadores**; `engine/conditions.py::matches` soporta **13**. Los `when` de los pasos y transiciones usan el segundo; las reglas de `rules.evaluate` usan el primero. `gte`, `lte`, `exists`, `not_exists`, `truthy`, `falsy` y `regex` **no funcionan dentro de `rules.evaluate`**. Duplicación registrada en 15.

`vision.inspect_screen_target` combina OCR y visión multimodal con tres modos (`ocr`, `vision`, `hybrid`), un `prefer_source` y un `fallback_bbox`. Devuelve `target_found`, `target_bbox`, `selected_source`, `decision` (`click` o `recover`) y `diagnostics[]`. Los errores de cada fuente **no abortan**: se acumulan en `diagnostics`. Es el único camino hacia `VisionModelAnalyzer` y ningún flow lo usa. Ver 09 §5.

2. Funciones centrales del motor

`Orchestrator.__init__(flow_dir, context_path=None, context_overrides=None)`

- **Efectos secundarios:** llama a `init_db()` (crea `db/runs.db` y sus 7 tablas), lee el manifest y el contexto, construye la `SandboxPolicy`, genera el `run_id` y crea el `JsonLogger` y el `StateStore` —lo que **crea los directorios `logs/` y `state/`**.
- **`run_id`:** `datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')`, con microsegundos. Es la clave primaria de `runs`. **INFERENCIA:** una colisión exigiría dos corridas en el mismo microsegundo.
- **Llamada por:** `app/server.py::do_POST`, `engine/runner.py::main`, `engine/scheduler.py::_run_job`, `scripts/smoke_test.py`.
- **Riesgo al modificar:** **crítico**. Es el constructor de todo el sistema.

`Orchestrator.run() -> dict`

- **Retorno:** el `state` completo con `status` (`completed` o `failed`), `steps[]`, `route[]`, `outputs[]`, `duration_seconds`.
- **Excepciones:** `FlowExecutionError` en violación de sandbox, acción no registrada, paso inexistente, exceso de `max_steps_per_run`, exceso de `max_runtime_seconds`, o fallo del último reintento sin rama de recuperación.
- **Efectos secundarios:** escribe en SQLite (`runs`, `steps`, `events`), en `logs/*.jsonl` y en `state/*.json` **tras cada paso**.
- **Riesgo al modificar:** **crítico**. 139 líneas de bucle con siete puntos de salida.

`SandboxPolicy.assert_paths_allowed(params)`

```
def _path_strings(self, params: Any) -> Iterable[str]
```

Recorre los parámetros recursivamente y considera candidata a ruta **cualquier cadena cuya clave contenga** `path`, `destination`, `source`, `output` o `file`. Descarta las que contienen `{` (placeholders sin resolver). Resuelve con `Path.resolve()` y comprueba `relative_to` contra cada base de `allowed_paths`.

Consecuencia no obvia: el filtro es por **nombre de clave**. Un parámetro que sea una ruta pero se llame `target` (como `browser.extract_content(target=...)`) **no se comprueba**. Es coherente con el catálogo actual, pero un autor de flows debe saberlo. Registrado en 11 · Seguridad.

`render_value(value, context)`

Recursiva sobre dict, list y str. Para una cadena:

1. Convierte `{{ → { y }} → }`.
2. Aplana el contexto (`flatten_context` genera claves compuestas `a.b.c`).
3. Añade `now` con formato `%Y%m%d_%H%M%S`.
4. Si la cadena es **exactamente** un placeholder, devuelve el valor **con su tipo original** (int, bool, dict, `None`).
5. Si no, sustituye cada `{clave}` presente y **deja literal** la que no exista.

Los dos últimos puntos son la clave. Un manifest que escribe `"headless": "{ { headless } }"` recibe el booleano real, no la cadena `"True"`. Y una llave que no es placeholder (JSON embebido en un comando) queda intacta, a diferencia de `str.format_map`, que crashearía. El propio módulo lo documenta.

Riesgo al modificar: **crítico**. Los 27 manifests dependen de esta semántica.

`evaluate_condition(condition, context)` y `matches(actual, operator, expected)`

`evaluate_condition` soporta los combinadores `all`, `any`, `not` de forma recursiva, y en la hoja usa `path` + `operator` + `value`. **Una condición vacía o `None` devuelve `True`** — es lo que permite que un paso sin `when` se ejecute siempre.

Los 13 operadores de `matches`:

Operador	Semántica
----------	-----------

<code>eq / ne</code>	Igualdad / desigualdad
<code>gt / gte / lt / lte</code>	Comparación; <code>None</code> siempre da <code>False</code>
<code>contains</code>	<code>str(expected).lower() in str(actual).lower()</code> — sin distinguir mayúsculas
<code>in</code>	<code>actual in expected</code> , solo si <code>expected</code> es lista
<code>exists / not_exists</code>	<code>actual is not None / is None</code>
<code>truthy / falsy</code>	<code>bool(actual) / not bool(actual)</code>
<code>regex</code>	<code>re.search(str(expected), str(actual))</code>

Un operador desconocido levanta `ValueError`, que el orquestador convierte en fallo del paso.

`LazyActionRegistry.get(action_name) -> ActionFn | None`

Cachea por nombre. Si no está en `_action_paths`, carga los entry points del grupo `automa.actions` **una sola vez** y reintenta. Devuelve `None` si sigue sin aparecer, y el orquestador lo convierte en `FlowExecutionError('Acción no registrada: ...')`.

Riesgo al modificar: `register_callable` guarda un `dotted_path` falso (`<inline>:nombre`) y mete la función directo en caché. Es el mecanismo que usan los tests.

`engine.paths.root_dir()` y `data_dir()`

Función	Orden de resolución
<code>root_dir()</code>	<code>\$AUTOMA_ROOT</code> → <code>sys._MEIPASS</code> → carpeta padre del archivo
<code>data_dir()</code>	<code>\$AUTOMA_DATA_ROOT</code> → si no está congelado, <code>root_dir()</code> → <code>%LOCALAPPDATA%\Automa</code> (Windows) o <code>\$XDG_DATA_HOME/automa-pc</code>

`data_dir()` crea el directorio si no existe. Separar ambas raíces fue el arreglo de la v0.2.1 al `PermissionError` bajo `Program Files`.

3. Endpoints HTTP

Servidor: `ThreadingHTTPServer` en `127.0.0.1:8787`.

3.1 GET — sin autorización

Ruta	Respuesta	Notas
<code>/</code>	HTML	Panel de 3 pestañas
<code>/healthz</code>	<code>{"status": "ok"}</code>	Sonda de vida
<code>/metrics</code>	<code>text/plain; version=0.0.4</code>	Formato Prometheus
<code>/api/metrics</code>	<code>{"overview": ..., "by_flow": [...]}</code>	JSON completo
<code>/metrics/dashboard</code>	HTML	Dashboard visual
<code>/api/flows</code>	<code>[{folder, id, name, family, description, steps, flow_path}]</code>	Catálogo
<code>/api/runs?flow_id=&limit=50</code>	Lista de corridas	Incluye <code>context_json</code> completo

<code>/api/runs/<run_id>/status</code>	Estado paso a paso	Para el polling del panel
<code>/flow/<folder></code>	HTML	Ficha del flow
<code>/flow/<folder>/config</code>	HTML	Editor de contexto
<code>/flow/<folder>/history</code>	HTML	Últimas corridas
<code>/run/<flow_id>/<run_id></code>	HTML	Detalle de una corrida
<code>/file?path=<relativa></code>	Bytes del archivo	Con cuatro controles; ver §3.3

Ningún GET exige token, ni siquiera con `AUTOMA_PANEL_TOKEN` definido.

3.2 POST — con autorización

Ruta	Cuerpo	Respuesta	Autorización
<code>/api/run/<folder></code>	<code>{"context_overrides": {...}}</code> opcional	<code>{ok, run_id, status, flow_id}</code> — asíncrono	<code>_authorize_mutation</code>
<code>/api/hook/<folder></code>	—	<code>{ok, run_id, status, flow_id}</code> — síncrono	<code>AUTOMA_WEBHOOK_TOKEN</code> obligatorio
<code>/api/form/submit</code>	JSON libre	<code>{ok, saved_path}</code>	<code>_authorize_mutation</code>
<code>/run?flow=<folder></code>	Formulario	Redirección 303 al detalle	<code>_authorize_mutation</code>
<code>/flow/<folder>/config</code>	<code>config_json=...</code>	HTML	<code>_authorize_mutation</code>
<code>/flow/<folder>/schedule</code>	<code>enabled, interval_seconds, cron_expression</code>	Redirección 303	<code>_authorize_mutation</code>

Códigos de estado: 200 OK · 303 redirección · 400 folder o JSON inválido · 401 no autorizado · 404 flow o ruta inexistente · 409 flow en preview · 415 extensión no servida por `/file` · 500 fallo de ejecución.

3.3 Los cuatro controles de `/file`

En orden, tal como están en `do_GET`:

1. Rechaza cadenas con `\x00` o cualquier carácter de control (`ord(c) < 32`).
2. Rechaza rutas absolutas: el cliente pide siempre relativas a la raíz.
3. `os.path.normpath(os.path.join(base_path, rel))` debe empezar por `base_path + os.sep` — el `os.sep` cierra el bypass por prefijo hermano (`/repo-evil` frente a `/repo`).
4. Allowlist negativa de extensiones: `.html`, `.htm`, `.xhtml`, `.xml`, `.svg`, `.js`, `.mjs`, `.css` devuelven 415. Cierra XSS reflejado desde el mismo origen.

Además envía `X-Content-Type-Options: nosniff`. Cubierto por `tests/test_security_hardening.py`.

3.4 `_safe_folder` — la primera línea de defensa

```
_FOLDER_RE = re.compile(r'^[A-Za-z0-9_-]{1,64}$')
```

Todo segmento `<folder>` de una URL pasa por aquí antes de tocar el catálogo o el filesystem. Rechaza `..`, separadores, NUL y no-ASCII. Se aplica en las siete rutas que reciben un folder.

4. Comandos

Comando	Qué hace	Código de salida
---------	----------	------------------

<code>automa list</code>	Catálogo en JSON	0, salvo <code>UnicodeEncodeError</code> en consola <code>cp1252</code>
<code>automa run <flow_dir> [--context RUTA]</code>	Ejecuta un flow e imprime el estado	0 / traza si falla
<code>automa scheduler [--interval N]</code>	Bucle del scheduler	No retorna
<code>automa-panel</code>	Panel HTTP	No retorna
<code>automa-desktop [--host --port --title --width --height --fullscreen]</code>	Ventana nativa	0 · 1 si el servidor no responde · 2 si falta <code>pywebview</code>
<code>automa-validate</code>	Valida los 27 manifests	0 si <code>ok</code> , 1 si hay errores
<code>python scripts/smoke_test.py</code>	Tres flows de punta a punta	0 / <code>AssertionError</code>
<code>python scripts/build_docs_pdf.py [--only N] [--check] [--out DIR] [--no-consolidado]</code>	Genera los PDF	0 / 1
<code>make test / lint / validate / smoke / list / run-panel / flow-health / clean</code>	Atajos	Según el comando

5. Variables de entorno

Variable	Leída por	Efecto si falta
<code>AUTOMA_PANEL_TOKEN</code>	<code>app/server.py::_authorize_mutation</code> vía <code>get_secret</code>	Se aplica el modo anti-CSRF por <code>Host/Origin/Referer</code>
<code>AUTOMA_WEBHOOK_TOKEN</code>	<code>app/server.py::_check_webhook_token</code>	<code>POST /api/hook/*</code> responde 401 siempre
<code>AUTOMA_ROOT</code>	<code>engine/paths.py::_root_dir</code>	Se usa la carpeta padre del paquete, o <code>sys._MEIPASS</code>
<code>AUTOMA_DATA_ROOT</code>	<code>engine/paths.py::_data_dir</code>	<code>root_dir()</code> en desarrollo, <code>%LOCALAPPDATA%\Automa</code> congelado
<code>OPENAI_API_KEY</code>	<code>VisionModelAnalyzer._analyze_openai_compatible</code>	Se envía la petición sin <code>Authorization</code> . Sin uso en el catálogo
<code>PYTHONIOENCODING</code>	Python	Recomendada en <code>utf-8</code> para el CLI en Windows

Ambos tokens se resuelven con `engine.secrets.get_secret`, así que también pueden vivir en `secrets/secrets.json` (ignorado por git) además de en el entorno.

`engine.secrets.list_secret_names()` expone los **nombres** conocidos —los del archivo, más las variables de entorno que empiecen por `AUTOMA_` o terminen en `_API_KEY` o `_TOKEN`— **sin exponer los valores**.

6. Campos del manifest

Contrato completo en `schemas/manifest.schema.json` (`additionalProperties: false` en la raíz y en cada paso: un campo desconocido es error de validación).

Campo	Tipo	Obligatorio	Por defecto
<code>id</code>	string, patrón <code>^[a-z0-9_]+\$</code>	[OK]	—

name	string no vacío	[OK]	—
steps	array con al menos 1 elemento	[OK]	—
description	string	—	""
family	string	—	"general"
start_step	string	—	Primer paso del array
max_steps_per_run	integer 1–10000	—	200
allowed_actions	array de strings únicos	—	null (sin restricción)
required_secrets	array de strings únicos	—	[]
allowed_paths	array de strings únicos	—	null (sin restricción)
max_runtime_seconds	number ≥ 0	—	null
preview	boolean	—	false

Campos de un paso: `id` [OK], `action` [OK], `params`, `save_as`, `retries` (0–100), `when`, `transitions`. **Campos de una transición:** `on` (`success`, `failure`, `any`), `next`, `end`, `when`.

7. Errores y excepciones

Excepción	Módulo	Cuándo
FlowExecutionError	engine/orchestrator.py	Violación de sandbox, acción no registrada, paso inexistente, exceso de pasos o de tiempo, fallo sin recuperación
SandboxViolation	engine/sandbox.py	Secreto ausente, acción bloqueada, ruta fuera de <code>allowed_paths</code>
CronExpressionError	engine/cron.py	Expresión con ≠ 5 campos, paso ≤ 0, rango fuera de límites, u horizonte de 4 años agotado
FileNotFoundError	Varias acciones	Ruta o imagen inexistente
FileExistsError	filesystem.move_file	Destino existente sin <code>overwrite</code>
ValueError	rules, conditions, ui.launch_process, system.run_powershell, vision.analyze_image	Operador o analizador desconocido, <code>shell=True</code> , token prohibido, verbo fuera de allowlist
RuntimeError	screen, ui, browser.*, vision_model_analyzer	Sin escritorio, sin <code>pyautogui</code> , sin Playwright, sin endpoint o modelo de visión
UnicodeEncodeError	engine/runner.py::main	Impresión de JSON no-ASCII en consola <code>cp1252</code> . Defecto conocido
sqlite3.IntegrityError	engine/database.py::acquire_run_lock	Capturada internamente: devuelve <code>False</code>

Kinds de error persistidos en `runs.error_json`: `sandbox_violation` (con `message` y, si aplica, `step_id`) o un objeto con `step_id` y `message` para los fallos de paso.

Eventos del log JSONL y de la tabla `events`: `flow_started`, `flow_finished`, `flow_blocked`, `step_started`, `step_finished`, `step_failed`, `step_skipped`, `step_blocked`, `step_recovered`.

Documentos relacionados: [04 · Mapa del código](#) · [06 · Explicación profunda](#) · [07 · Base de datos](#) · [09 · APIs e integraciones](#) · [10 · Configuración](#)